

Linux TCP 서버의 I/O 멀티플렉싱 성능 비교

+비동기 프로그래밍(select, epoll, io_uring)

김학재

2026.09.17

목차

1. 지난시간 (select, epoll 작동방식)
2. 리눅스 환경에서의 비동기 프로그래밍
3. io_uring
4. 성능비교

지난시간

한 서버가 여러 클라이언트의 통신을 동시에 처리하기 위해서

1) 멀티프로세싱 – fork

- 프로세스 생성에 많은 시간과 자원필요
- 프로세스간의 통신 복잡

2) 멀티스레딩 – thread

- 스레드의 생성 및 컨텍스트 스위칭이 프로세스보다는 빠르고 가볍다.
- 스레드간의 통신은 덜 복잡 (메모리 공유해서)

3) 멀티플렉싱 – select() , epoll()

- 싱글 프로세스, 싱글 스레드로 여러 클라이언트와 통신 가능 (컨텍스트 스위칭 X)
- 멀티플렉싱은 엄밀히 말하면 여러 클라이언트와의 통신을 동시에 실행하지는 않는다는 점. (멀티코어인 경우)

4) 비동기 I/O 인터페이스 – io_uring()

- 싱글 프로세스, 싱글 스레드에서 여러 I/O작업 처리 가능
- I/O 요청을 제출하고 완료 결과를 나중에 확인하는 구조

select

select() : 비트맵으로 이루어져있는 fd_set을 감시하면서 조건 만족한 fd 총 개수를 반환, fd_set을 수정하는 함수.

정정할 내용

```
487 | retval = max_select_fd(n, fds);
```

fd가 1000번까지 있다고 칠 때 만약
fd 59, 125, 127번이 감시가 필요하다면 127번까지만 순회하도록
최대 어디까지 검사할 것인지 정해줌

127번까지 검사하는게 아니라 1000번까지 검사한다.
이벤트 발생에대한 번호까지만 검사한다는 최적화는 없었다.

fd

in 비트맵

out 비트맵

ex 비트맵

0	1	2	3	4	5	6	7
0	0	0	1	0	1	0	0
0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0
0	0	0	1	0	1	0	1

epoll

epoll() : 관심 있는 FD를 커널에 등록해 두고, 이벤트가 발생한 FD를 전달받는 방식

epoll_ctl() : Interest list에 등록/수정/삭제

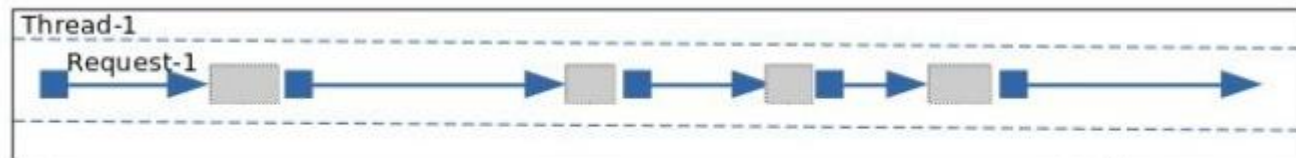


커널이 fd중 이벤트 발생한것에 대해 ready list에 넣음.

epoll_wait() :이벤트 정보 반환

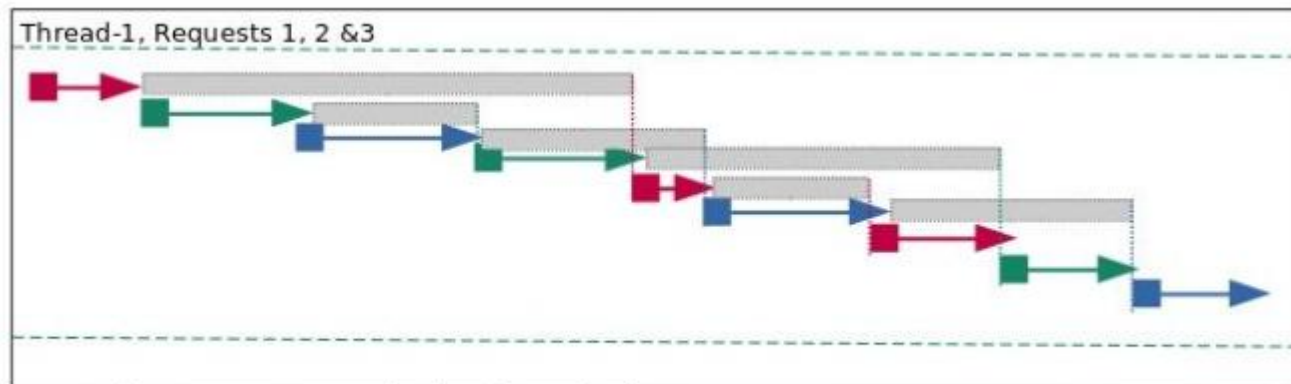
select는 이벤트 발생하면 짝 훑어서 누가발생했는지,
epoll은 이벤트 발생하면 ready list에 바로 넣는다.

비동기



Single threaded app

blocked

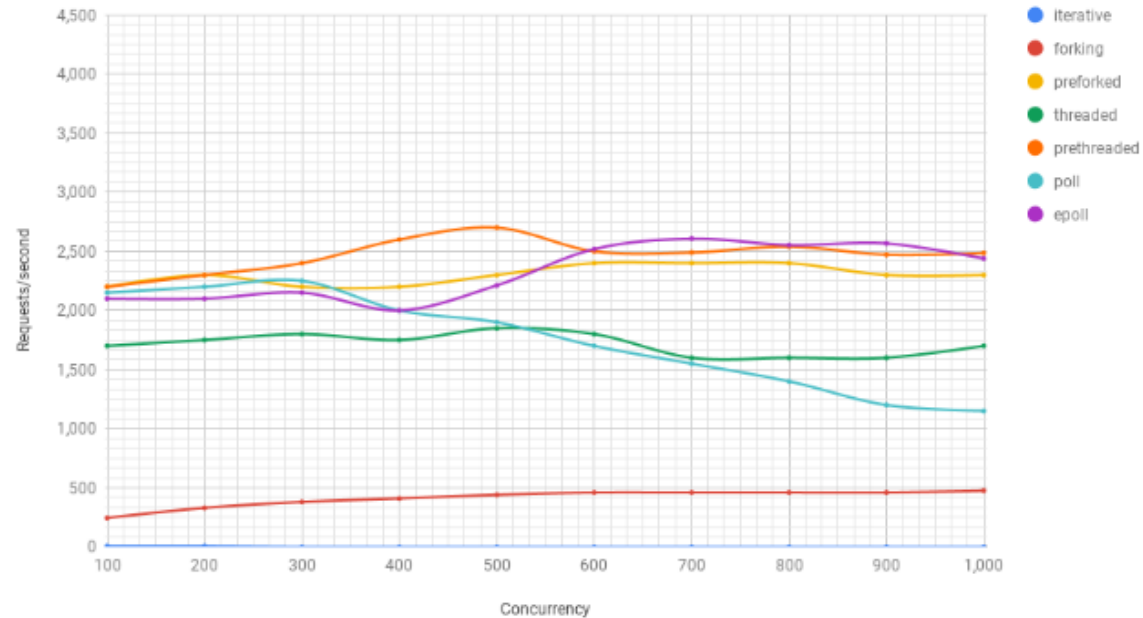


Asynchronous app; Single threaded.

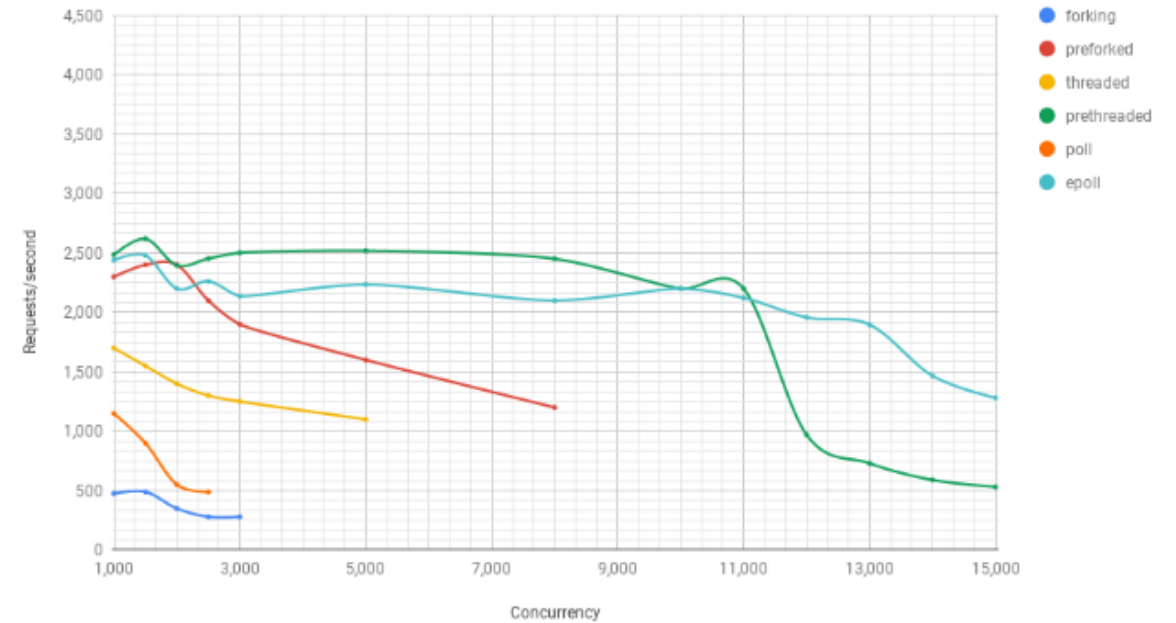
blocked

성능지표

Requests/sec Vs. Concurrency (Upto 1,000 concurrent connections)



Requests/sec Vs. Concurrency (1,000+ concurrent connections)



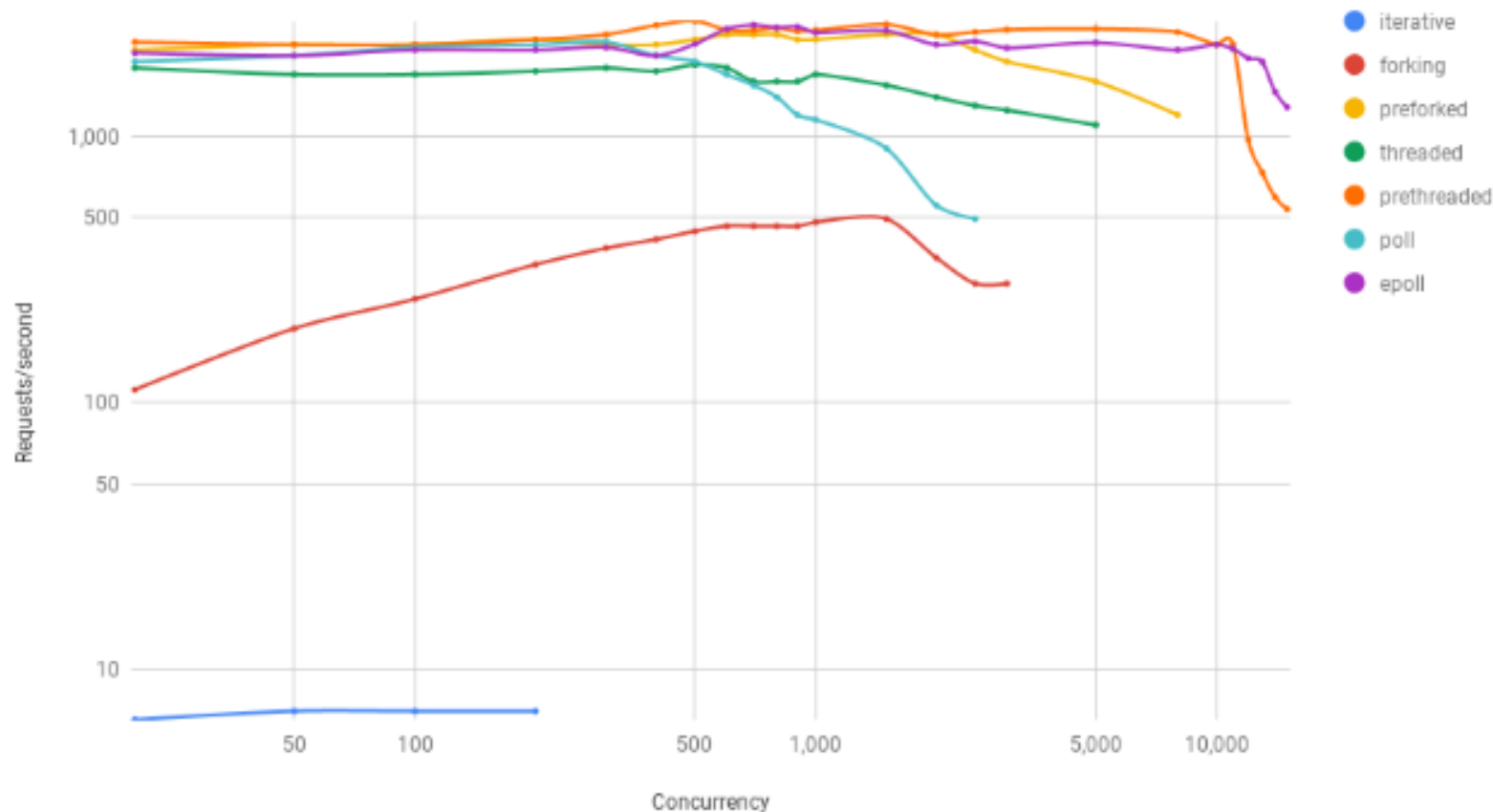
preforked, prethreaded : 서버가 시작될 때 프로세스나, 스레드 미리 만들어두는거임

forking, threaded : 요청 들어올때마다 만드는 것

초당 처리하는 요청의 개수, 동시성

Requests/second Vs. Concurrency

Note: Both axis as in log scale



<순위>
epoll
prethreaded
preforked
threaded
poll
forking
iterative

리눅스 환경에서의 비동기 프로그래밍

일반적으로 read와 같은 시스템

호출을 실행하면 파일이 읽히고 데이터가 사용 가능해질 때까지 프로그램이 차단된다.

(기본은 blocking 하지만 non-blocking으로도 설정가능)

blocking은 데이터가 올 때까지 기다림

non-blcking은 데이터 있든 없든 즉시 반환

예를 들어 클라가 1000명일 때 blocking같은 동기프로그래밍으로 통신을하면 한명때문에 다 막힐 수 있음.

select / epoll 이런 애들은 여러 fd의 I/O준비 상태를 확인하고 알려주는 애들이다

소켓과 일반 파일은 다르게 동작한다

리눅스에서 파일 디스크립터(fd)에 일반 파일, 소켓, 파이프 등이 포함되는데 select, epoll 은 일반 파일의 디스크 I/O 완료 기다리는 용도로 사용할 수 없다.

왜냐하면 일반 파일은 이러한 준비상태 감시하는 select, epoll 한테는 즉시 준비되어있는 상태로 보고될 수 있다.

이것이 의미하는게 다른게

소켓에서 준비 됐다는 것은 커널 버퍼에 읽을 데이터가 도착해 있을 가능성이 높지만,

파일에서 준비 됐다는 것은 아직 디스크에서 메모리로 꺼내온 상태는 아니라서 이게 얼마나 걸릴지는 모른다.

io_uring()

파일과 소켓 모두에 대해 통일된 인터페이스로 비동기 I/O 작업을 제출하고 완료결과를 확인할 수 있도록 설계된 리눅스 I/O 인터페이스

I/O 작업을 커널에 요청하고, 작업이 완료되면 완료 결과를 받는 방식.

Submission Queue (SQ) -> 커널 -> Completion Queue (CQ)

io_uring_enter()라는 함수로 SQ처리하게 시킴.

그럼 한번에 처리를 함으로 syscall 횟수가 줄어듦

sq,cq 둘 다 ring buffer로 이루어져있고 kernel과 user space가 공유한다. (접근가능)

select_server vs epoll_server

1000개 연결 후 N개만 통신했을 때

select

```
=====
BENCHMARK RESULT
=====
total clients : 1000
active clients : 10
messages/client: 1000
message size : 1024 bytes
total messages : 10000
total bytes : 10240000
elapsed : 0.203752 sec
messages/sec : 49079.35
MB/sec : 47.93
=====
```

```
=====
BENCHMARK RESULT
=====
total clients : 1000
active clients : 100
messages/client: 1000
message size : 1024 bytes
total messages : 100000
total bytes : 102400000
elapsed : 0.764220 sec
messages/sec : 130852.35
MB/sec : 127.79
=====
```

epoll

```
=====
BENCHMARK RESULT
=====
total clients : 1000
active clients : 10
messages/client: 1000
message size : 1024 bytes
total messages : 10000
total bytes : 10240000
elapsed : 0.067387 sec
messages/sec : 148395.74
MB/sec : 144.92
=====
```

```
=====
BENCHMARK RESULT
=====
total clients : 1000
active clients : 100
messages/client: 1000
message size : 1024 bytes
total messages : 100000
total bytes : 102400000
elapsed : 0.652015 sec
messages/sec : 153370.82
MB/sec : 149.78
=====
```

io_uring

```
=====
BENCHMARK RESULT
=====
total clients : 1000
active clients : 10
messages/client: 1000
message size : 1024 bytes
total messages : 10000
total bytes : 10240000
elapsed : 0.071129 sec
messages/sec : 140590.32
MB/sec : 137.30
=====
```

```
=====
BENCHMARK RESULT
=====
total clients : 1000
active clients : 100
messages/client: 1000
message size : 1024 bytes
total messages : 100000
total bytes : 102400000
elapsed : 0.662384 sec
messages/sec : 150969.80
MB/sec : 147.43
=====
```

select_server vs epoll_server

1000개 연결 후 (500개,1개)만 통신했을 때

select

epoll

```
=====
BENCHMARK RESULT
=====
total clients   : 1000
active clients  : 500
messages/client: 1000
message size    : 1024 bytes
total messages  : 500000
total bytes     : 512000000
elapsed        : 3.287627 sec
messages/sec    : 152085.36
MB/sec         : 148.52
=====
```

```
=====
BENCHMARK RESULT
=====
total clients   : 1000
active clients  : 500
messages/client: 1000
message size    : 1024 bytes
total messages  : 500000
total bytes     : 512000000
elapsed        : 3.280898 sec
messages/sec    : 152397.31
MB/sec         : 148.83
=====
```

```
=====
BENCHMARK RESULT
=====
total clients   : 1000
active clients  : 1
messages/client: 1000
message size    : 1024 bytes
total messages  : 1000
total bytes     : 1024000
elapsed        : 0.285289 sec
messages/sec    : 3505.22
MB/sec         : 3.42
=====
```

```
=====
BENCHMARK RESULT
=====
total clients   : 1000
active clients  : 1
messages/client: 1000
message size    : 1024 bytes
total messages  : 1000
total bytes     : 1024000
elapsed        : 0.037832 sec
messages/sec    : 26432.90
MB/sec         : 25.81
=====
```

감사합니다